

```
using System;
using System.Diagnostics;
namespace consoleblocking
{
    class MainClass
    {
        public static void Main (string[] args)
        {
            Console.WriteLine ("Non-blocking");
            Process.Start (".\\shell-block.sh");
            Console.WriteLine ("Blocking");
            ProcessStartInfo startInfo = new ProcessStartInfo();
            startInfo.FileName = ".\\shell-block.sh";
            startInfo.Arguments = "";
            using (var myProcess = Process.Start(startInfo))
            {
                myProcess.WaitForExit();
            }

            List<Student> studentList = new List<Student>() {
                new Student () { StudentId="001", StudentName="admin", StudentSex="M"},
                new Student () { StudentId="002", StudentName="test",
StudentSex="M"},
                new Student () { StudentId="003", StudentName="development",
StudentSex="M"},
                new Student () { StudentId="004", StudentName="product",
StudentSex="M"},
                new Student () { StudentId="005", StudentName="saler",
StudentSex="M"},
                new Student () { StudentId="001", StudentName="admin",
StudentSex="M"},
                new Student () { StudentId="002", StudentName="test",
StudentSex="F"},
                new Student () { StudentId="003", StudentName="development",
StudentSex="F"},
                new Student () { StudentId="004", StudentName="product",
StudentSex="F"},
                new Student () { StudentId="005", StudentName="saler",
StudentSex="F"},
            };

            class StudentCompare : IEqualityComparer<Student>
            {
                public bool Equals(Student x, Student y)
                {
                    if (x.StudentId == y.StudentId && x.StudentName == y.StudentName)
                    {
                        return true;
                    }
                    else
                    {
                        return false;
                    }
                }
            }
        }
    }
}
```

```

    }
}
public int GetHashCode(Student obj)
{
    if (obj == null)
    {
        return 0;
    }
    return obj.ToString().GetHashCode();
}
}

public static void DistinctList(List<Student> orgianList)
{
    List<Student>      returnList      =      orgianList.Distinct(new
StudentCompare()).ToList();
    Console.WriteLine(returnList.Count);
    returnList.ForEach(c =>
    {
        Console.WriteLine(c.StudentId + "\t" + c.StudentName + "\t" +
c.StudentSex);
    });
}

public bool Equals(T x, T y)
{
    var firstType = x.GetType();
    var secondType = y.GetType();
    if (firstType == secondType)
    {
        var firstProperty = firstType.GetProperties();
        var secondProperty = secondType.GetProperties();
        foreach (var first in firstProperty)
        {
            foreach (var second in secondProperty)
            {
                if (first.Name == second.Name)
                {
                    var fistValue = first.GetValue(x);
                    var secondValue = second.GetValue(y);
                    Console.WriteLine("    【  fistValue  】    :{0}    \t
【secondValue】:{1}", fistValue, secondValue);
                    if (fistValue == secondValue)
                    {
                        return true;
                    }
                }
                else
                {
                    return false;
                }
            }
        }
    }
}
}

```

```

    }
}
return false;
}

private void AssetCodePrint_Load(object sender, EventArgs e)
{
    LoadCmb(this._cmb, SqlDb.Executedt(string.Format("SELECT
GroupCode,Description FROM Erp.FAGroup WHERE Company='{0}' ORDER BY Description
collate Chinese_PRC_CI_AS", MyData.Session.CompanyID)), "GroupCode",
"Description");
    LoadCmb(this._cmb, SqlDb.Executedt(string.Format("SELECT
JCDept,Description FROM JCDept WHERE Company='{0}' ORDER BY Description collate
Chinese_PRC_CI_AS", MyData.Session.CompanyID)), "JCDept", "Description");
    if (strAssetNum.Trim() != "")
    {
        this._txt.Text = strAssetNum;
        btnFind_Click(null, null);
        this.chkAll.Checked = true;
        btnPrint_Click(null, null);
    }
    this.Icon = new Icon(Application.StartupPath + @"\epicor.ico", 16,
16);
}

private void LoadCmb(ComboBox combo, DataTable CmbData, string ValueMember,
string DisplayMember)
{
    combo.DataSource = CmbData;
    combo.ValueMember = ValueMember;
    combo.DisplayMember = DisplayMember;
    combo.SelectedIndex = -1;
}

public static string GetMd5Hash( string input)
{
    Convert the input string to a byte array and compute the hash.
    byte[] data =
MD5.Create().ComputeHash(Encoding.UTF8.GetBytes(input));
    Create a new StringBuilder to collect the bytes
    and create a string.
    StringBuilder sBuilder = new StringBuilder();
    Loop through each byte of the hashed data
    and format each one as a hexadecimal string.
    for (int i = 0; i < data.Length; i++)
    {
        sBuilder.Append(data[i].ToString("x2"));
    }
    return sBuilder.ToString();
}

public static void BulkInsert<T>(string connection ,string tableName,
IList<T> list)
{

```

```

        using (var bulkCopy = new SqlBulkCopy(connection))
        {
            bulkCopy.BatchSize = list.Count;
            bulkCopy.DestinationTableName = tableName;
            var table = new DataTable();
            var props = TypeDescriptor.GetProperties(typeof(T))
                                    .Cast<PropertyDescriptor>()
                                    .Where(propertyInfo =>
propertyInfo.PropertyType.Namespace.Equals("System"))
                                    .ToArray();
            foreach (var propertyInfo in props)
            {
                bulkCopy.ColumnMappings.Add(propertyInfo.Name,
propertyInfo.Name);
                table.Columns.Add(propertyInfo.Name,
Nullable.GetUnderlyingType(propertyInfo.PropertyType) ??
propertyInfo.PropertyType);
            }
            var values = new object[props.Length];
            foreach (var item in list)
            {
                for (var i = 0; i < values.Length; i++)
                {
                    values[i] = props[i].GetValue(item);
                }
                table.Rows.Add(values);
            }
            bulkCopy.WriteToServer(table);
        }
    }

    public class ClassJournal : ICloneable, IClassJournal
    {
        #region Member Variables
        protected int _cjid;
        protected int? _ctid;
        protected int? _studentcount;
        protected int? _girlcount;
        protected int? _minoritycount;
        protected string _monitorname;
        protected string _classhonorary;
        protected bool _bIsDeleted;
        protected bool _bIsChanged;
        #endregion
        #region Constructors
        public ClassJournal() {}
        public ClassJournal(int? pCtId, int? pStudentCount, int? pGirlCount, int?
pMinorityCount, string pMonitorName, string pClassHonorary)
        {
            this._ctid = pCtId;
            this._studentcount = pStudentCount;

```

```
this._girlcount = pGirlCount;
this._minoritycount = pMinorityCount;
this._monitorname = pMonitorName;
this._classhonorary = pClassHonorary;
}
#endregion
#region Public Properties
public virtual int CjId
{
    get { return _cjid; }
    set { _bIsChanged |= (_cjid != value); _cjid = value; }
}
public virtual int? CtId
{
    get { return _ctid; }
    set { _bIsChanged |= (_ctid != value); _ctid = value; }
}
public virtual int? StudentCount
{
    get { return _studentcount; }
    set { _bIsChanged |= (_studentcount != value); _studentcount = value; }
}
public virtual int? GirlCount
{
    get { return _girlcount; }
    set { _bIsChanged |= (_girlcount != value); _girlcount = value; }
}
public virtual int? MinorityCount
{
    get { return _minoritycount; }
    set { _bIsChanged |= (_minoritycount != value); _minoritycount = value; }
}
public virtual string MonitorName
{
    get { return _monitorname; }
    set
    {
        if (value != null && value.Length > 10)
            throw new ArgumentOutOfRangeException("MonitorName", "MonitorName value,
cannot contain more than 10 characters");
        _bIsChanged |= (_monitorname != value);
        _monitorname = value;
    }
}
public virtual string ClassHonorary
{
    get { return _classhonorary; }
    set
    {
        if (value != null && value.Length > 100)
```

```

        throw new ArgumentOutOfRangeException("ClassHonorary", "ClassHonorary value,
cannot contain more than 100 characters");
        _bIsChanged |= (_classshonorary != value);
        _classshonorary = value;
    }
}
#endregion
#region Equals And GetHashCode Overrides
public override bool Equals( object obj )
{
    if( this == obj ) return true;
    ClassJournal castObj = null;
    try
    {
        castObj = (ClassJournal)obj;
    } catch(Exception) { return false; }
    return ( castObj != null ) &&
        ( this._cjid == castObj.CjId );
}
public override int GetHashCode()
{
    int hash = 57;
    hash = 27 * hash * _cjid.GetHashCode();
    return hash;
}
#endregion
#region ICloneable methods
public virtual object Clone()
{
    return this.MemberwiseClone();
}
#endregion
#region Custom ICollection interface for ClassJournal
public interface ICollection : ICollection
{
    ClassJournal this[int index]{get; set; }
    void Add(ClassJournal pClassJournal);
    void Clear();
}
[Serializable]
public class ClassJournalCollection : ICollection
{
    private IList<ClassJournal> _arrayInternal;
    public ClassJournalCollection()
    {
        _arrayInternal = new List<ClassJournal>();
    }
    public ClassJournalCollection( IList<ClassJournal> pSource )

```

```

{
_arrayInternal = pSource;
if(_arrayInternal == null)
{
_arrayInternal = new List<ClassJournal>();
}
}

public ClassJournal this[int index]
{
get
{
return _arrayInternal[index];
}
set
{
_arrayInternal[index] = value;
}
}

public int Count { get { return _arrayInternal.Count; } }
public bool IsSynchronized { get { return false; } }
public object SyncRoot { get { return _arrayInternal; } }
public void CopyTo(Array array, int index) { _arrayInternal.CopyTo((ClassJournal[])array, index); }
public IEnumerator GetEnumerator() { return _arrayInternal.GetEnumerator(); }
public void Add(ClassJournal pClassJournal) { _arrayInternal.Add(pClassJournal); }
public void Clear() { _arrayInternal.Clear(); }
public IList<ClassJournal> GetList() { return _arrayInternal; }
}

#endregion
}

version: 1.0.0. {build}
image:
- Visual Studio 2017
pull_requests:
do_not_increment_build_number: true
#branches:
# only:
# - master
# - Developer
skip_branch_with_pr: true
configuration: Release
before_build:
- cmd: dotnet restore
Samples\Senparc.Weixin.MP.Sample.vs2017\Senparc.Weixin.MP.Sample.vs2017.sln
build:
verbosity: minimal
parallel: true
build_script:
- cmd: dotnet build -c Release
Samples\Senparc.Weixin.MP.Sample.vs2017\Senparc.Weixin.MP.Sample.vs2017.sln

```

```

test_script:
  - cmd: dotnet test -c Release
    Samples\Senparc.Weixin.MP.Sample.vs2017\Senparc.Weixin.MP.Sample.vs2017.sln
artifacts:
  - path: '**\*.nupkg'
    name: nupkg
  - path: '**\*.snupkg'
    name: snupkg
nuget:
  disable_publish_on_pr: true
private string[] namelist = new string[size];
static public int size = 10;
public string this[int index]
{
    get
    {
        string tmp;
using System.Collections.Generic;
using System.Linq;
using System.Reflection;
namespace KuroLib.Helper
{
    public static class ReflexHelper
    {
        public static T GetField<T>(this object instance, string fieldname)
        {
            public class GetMousePos : MonoBehaviour
        }
    }
}

public Canvas canvas;
private RectTransform rectTransform;
}
void Start()
{
    canvas = GameObject.Find("Canvas").GetComponent<Canvas>();
    rectTransform = canvas.transform as RectTransform;
}
class Program {
static void Main(string[] args) {
    T t1 = new T(4);
    T t2 = new T(4);
    Console.WriteLine(": {0}", t1.CompareTo(t2));
    List<int> array = new List<int>();
    for (int i = 1; i <= 4; i++) {
        for (int j = 0; j < i; j++) {
            public partial class Form1 : Form
        {
            const short WM_CAP = 1024;
            const int WM_CAP_DRIVER_CONNECT = WM_CAP + 10;
            const int WM_CAP_DRIVER_DISCONNECT = WM_CAP + 11;

```



```

        const int WM_CAP_EDIT_COPY = WM_CAP + 30;
        const int WM_CAP_SET_PREVIEW = WM_CAP + 50;
        const int WM_CAP_SET_PREVIEWRATE = WM_CAP + 52;
        const int WM_CAP_SET_SCALE = WM_CAP + 53;
        const int WS_CHILD = 1073741824;
    }

    using System;
    using System.Collections.Generic;
    using System.Linq;
    using System.Text;
    using System.Threading.Tasks;
    namespace Second
    {
        public delegate void ShengJi(); }
        public class FileLogger
        {
            #region
            public static object _lock = new object();
            public static int day = 30;
            public static string GetMacByNetworkInterface()
        {
            List<string> macs = new List<string>();
            NetworkInterface[] interfaces = NetworkInterface.GetAllNetworkInterfaces();
            return interfaces.Where(p => p.OperationalStatus ==
            OperationalStatus.Up).FirstOrDefault().GetPhysicalAddress().ToString();
            using Microsoft.Web.Administration;
        }
        using System;
        using System.Collections.Generic;
        using System.DirectoryServices;
        using System.Linq;
        using System.Text;
        using System.Threading.Tasks;
        namespace Network.Housekeeper.Core.IISManger
        {
            private void lblTitle_MouseDown(object sender, MouseEventArgs e)
            {
                beginMove = true;
                currentXPosition = MousePosition.X;
                currentYPosition = MousePosition.Y;
                this.Refresh();
            }
        }
    }

    ## Ignore Visual Studio temporary files, build results, and
    ## files generated by popular Visual Studio add-ons.
    # User-specific files
    *.suo
    *.user
    *.userosscache
    *.sln.docstates
    # User-specific files (MonoDevelop/Xamarin Studio)

```

```
*.userprefs
# Build results
[Dd]ebug/
[Dd]ebugPublic/
[Rr]elease/
[Rr]eleases/
x64/
x86/
bld/
[Bb]in/
[Oo]bj/
[Ll]og/
# Visual Studio 2015 cache/options directory
.vs/
# Uncomment if you have tasks that create the project's static files in wwwroot
#wwwroot/
# MSTest test Results
[Tt]est[Rr]esult*/
[Bb]uild[Ll]og.*
# NUNIT
*.VisualState.xml
TestResult.xml
# Build Results of an ATL Project
[Dd]ebugPS/
[Rr]eleasePS/
dlldata.c
# .NET Core
project.lock.json
project.fragment.lock.json
artifacts/
private string[] namelist = new string[size];
static public int size = 10;
public string this[int index]
{
    get
    {
        string tmp;
        if (index >= 0 && index <= size - 1)
        {
            tmp = namelist[index];
        }
        else
        {
            tmp = "";
        }
        return (tmp);
    }
    set
    {
        if (index >= 0 && index <= size - 1)
```

```

        {
            namelist[index] = value;
        }
    }
}

IndexedNames names = new IndexedNames();
names[0] = "Zara";
names[1] = "Riz";
names[2] = "Nuha";
names[3] = "Asif";
using System.Collections.Generic;
using System.Linq;
using System.Reflection;
namespace KuroLib.Helper
{
    public static class ReflexHelper
    {
        public static T GetField<T>(this object instance, string fieldname)
        {
            var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.NonPublic | BindingFlags.Public;
            var type = instance.GetType();
            var field = type.GetField(fieldname, flag);
            return (T)field?.GetValue(instance);
        }

        public static T GetPrivateField<T>(this object instance, string fieldname)
        {
            var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.NonPublic;
            var type = instance.GetType();
            var field = type.GetField(fieldname, flag);
            return (T)field?.GetValue(instance);
        }

        public static T GetPublicField<T>(this object instance, string fieldname)
        {
            var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.Public;
            var type = instance.GetType();
            var field = type.GetField(fieldname, flag);
            return (T)field?.GetValue(instance);
        }

        public static List<FieldInfo> GetFieldList<T>(this object instance)
        {
            var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.NonPublic | BindingFlags.Public;
            var type = instance.GetType();
            return type.GetFields(flag).ToList();
        }

        public static List<FieldInfo> GetPrivateFieldList<T>(this object
instance)

```

```

        {
            var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.NonPublic;
            var type = instance.GetType();
            return type.GetFields(flag).ToList();
        }
        public static List<FieldInfo> GetPublicFieldList<T>(this object instance)
        {
            var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.Public;
            var type = instance.GetType();
            return type.GetFields(flag).ToList();
        }
        public static T GetFieldInBase<T>(this object instance, string fieldname)
        {
            var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.NonPublic | BindingFlags.Public;
            var type = instance.GetType().BaseType;
            var field = type?.GetField(fieldname, flag);
            return (T)field?.GetValue(instance);
        }
        public static T GetPrivateFieldInBase<T>(this object instance, string
fieldname)
        {
            var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.NonPublic;
            var type = instance.GetType().BaseType;
            var field = type?.GetField(fieldname, flag);
            return (T)field?.GetValue(instance);
        }
        public static T GetPublicFieldInBase<T>(this object instance, string
fieldname)
        {
            var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.Public;
            var type = instance.GetType().BaseType;
            var field = type?.GetField(fieldname, flag);
            return (T)field?.GetValue(instance);
        }
        public static List<FieldInfo> GetFieldListInBase<T>(this object instance)
        {
            var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.NonPublic | BindingFlags.Public;
            var type = instance.GetType().BaseType;
            return type?.GetFields(flag).ToList();
        }
        public static List<FieldInfo> GetPrivateFieldListInBase<T>(this object
instance)
        {
            var flag = BindingFlags.Instance | BindingFlags.Static |

```

```

BindingFlags.NonPublic;
        var type = instance.GetType().BaseType;
        return type?.GetFields(flag).ToList();
    }
    public static List<FieldInfo> GetPublicFieldListInBase<T>(this object
instance)
    {
        var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.Public;
        var type = instance.GetType().BaseType;
        return type?.GetFields(flag).ToList();
    }
    public static void SetField(this object instance, string fieldname, object
value)
    {
        var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.NonPublic | BindingFlags.Public;
        var type = instance.GetType();
        var field = type.GetField(fieldname, flag);
        field?.SetValue(instance, value);
    }
    public static void SetPrivateField(this object instance, string fieldname,
object value)
    {
        var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.NonPublic;
        var type = instance.GetType();
        var field = type.GetField(fieldname, flag);
        field?.SetValue(instance, value);
    }
    public static void SetPublicField(this object instance, string fieldname,
object value)
    {
        var flag = BindingFlags.Instance | BindingFlags.Public;
        var type = instance.GetType();
        var field = type.GetField(fieldname, flag);
        field?.SetValue(instance, value);
    }
    public static void SetFieldInBase(this object instance, string fieldname,
object value)
    {
        var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.NonPublic | BindingFlags.Public;
        var type = instance.GetType().BaseType;
        var field = type?.GetField(fieldname, flag);
        field?.SetValue(instance, value);
    }
    public static void SetPrivateFieldInBase(this object instance, string
fieldname, object value)
    {

```

```
        var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.NonPublic;
        var type = instance.GetType().BaseType;
        var field = type?.GetField(fieldname, flag);
        field?.SetValue(instance, value);
    }

    public static void SetPublicFieldInBase(this object instance, string
fieldname, object value)
    {
        var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.Public;
        var type = instance.GetType().BaseType;
        var field = type?.GetField(fieldname, flag);
        field?.SetValue(instance, value);
    }

    public static T GetPrivateProperty<T>(this object instance, string
propertyname)
    {
        var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.NonPublic;
        var type = instance.GetType();
        var field = type.GetProperty(propertyname, flag);
        return (T)field.GetValue(instance, null);
    }

    public static T GetPublicProperty<T>(this object instance, string
propertyname)
    {
        var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.Public;
        var type = instance.GetType();
        var field = type.GetProperty(propertyname, flag);
        return (T)field.GetValue(instance, null);
    }

    public static void SetPrivateProperty(this object instance, string
propertyname, object value)
    {
        var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.NonPublic;
        var type = instance.GetType();
        var field = type.GetProperty(propertyname, flag);
        field.SetValue(instance, value, null);
    }

    public static void SetPublicProperty(this object instance, string
propertyname, object value)
    {
        var flag = BindingFlags.Instance | BindingFlags.Public;
        var type = instance.GetType();
        var field = type.GetProperty(propertyname, flag);
        field.SetValue(instance, value, null);
    }
}
```

```

        public static T CallPrivateMethod<T>(this object instance, string name,
params object[] param)
        {
            var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.NonPublic;
            var type = instance.GetType();
            var method = type.GetMethod(name, flag);
            return (T)method.Invoke(instance, param);
        }
        public static T CallPublicMethod<T>(this object instance, string name,
params object[] param)
        {
            var flag = BindingFlags.Instance | BindingFlags.Public;
            var type = instance.GetType();
            var method = type.GetMethod(name, flag);
            return (T)method.Invoke(instance, param);
        }
    }
}
public class GetMousePos : MonoBehaviour
{
    public Canvas canvas;
    private RectTransform rectTransform;
    void Start()
    {
        canvas = GameObject.Find("Canvas").GetComponent<Canvas>();
        rectTransform = canvas.transform as RectTransform;
    }
    void Update()
    {
        if (Input.GetMouseButtonDown(0))
        {
            Vector2 pos;
            if
(RectTransformUtility.ScreenPointToLocalPointInRectangle(rectTransform,
Input.mousePosition, canvas.worldCamera, out pos))
            {
                rectTransform.anchoredPosition = pos;
                Debug.Log(pos);
            }
        }
    }
}
public class GetMousePos : MonoBehaviour
{
    public Canvas canvas;
    private RectTransform rectTransform;
    void Start()
    {
        canvas = GameObject.Find("Canvas").GetComponent<Canvas>();

```

```

        rectTransform = canvas.transform as RectTransform;

    }

    void Update()
    {
        if (Input.GetMouseButtonDown(0))
        {
            Vector2 pos;
            if
(RectTransformUtility.ScreenPointToLocalPointInRectangle(rectTransform,
Input.mousePosition, canvas.worldCamera, out pos))
            {
                rectTransform.anchoredPosition = pos;
                Debug.Log(pos);
            }
        }
    }
}

class Program {
    static void Main(string[] args) {
        T t1 = new T(4);
        T t2 = new T(4);
        Console.WriteLine(": {0}", t1.CompareTo(t2));

        List<int> array = new List<int>();
        for (int i = 1; i <= 4; i++) {
            for (int j = 0; j < i; j++) {
                array.Add(i);
            }
        }
        int one = 0, two = 0, three = 0, four = 0;
        foreach (int oneArray in array) {
            if (oneArray == 1) one++;
            if (oneArray == 2) two++;
            if (oneArray == 3) three++;
            if (oneArray == 4) four++;
        }
        Console.WriteLine("\"1\"{0}", one);
        Console.WriteLine("\"2\"{0}", two);
        Console.WriteLine("\"3\"{0}", three);
        Console.WriteLine("\"4\"{0}", four);
    }
}

class T : IComparable<T>{
    public int num;
    public T(int num) {
        this.num = num;
    }
    public int CompareTo(T other) {

```



```

        return this.num.CompareTo(other.num);
    }
}

public partial class Form1 : Form
{
    const short WM_CAP = 1024;
    const int WM_CAP_DRIVER_CONNECT = WM_CAP + 10;
    const int WM_CAP_DRIVER_DISCONNECT = WM_CAP + 11;
    const int WM_CAP_EDIT_COPY = WM_CAP + 30;
    const int WM_CAP_SET_PREVIEW = WM_CAP + 50;
    const int WM_CAP_SET_PREVIEWRATE = WM_CAP + 52;
    const int WM_CAP_SET_SCALE = WM_CAP + 53;
    const int WS_CHILD = 1073741824;
    const int WS_VISIBLE = 268435456;
    const short SWP_NOMOVE = 2;
    const short SWP_NOSIZE = 1;
    const short SWP_NOZORDER = 4;
    const short HWND_BOTTOM = 1;
    int iDevice = 0;
    int hHwnd;
    [System.Runtime.InteropServices.DllImport("user32", EntryPoint =
"SendMessageA")]
    static extern int SendMessage(int hwnd, int wMsg, int wParam,
[MarshalAs(UnmanagedType.AsAny)]
    object lParam);
    [System.Runtime.InteropServices.DllImport("user32", EntryPoint =
"SetWindowPos")]
    static extern int SetWindowPos(int hwnd, int hWndInsertAfter, int x, int
y, int cx, int cy, int wFlags);
    [System.Runtime.InteropServices.DllImport("user32")]
    static extern bool DestroyWindow(int hwnd);
    [System.Runtime.InteropServices.DllImport("avicap32.dll")]
    static extern int capCreateCaptureWindowA(string lpszWindowName, int
dwStyle, int x, int y, int nWidth, short nHeight, int hWndParent, int nID);
    [System.Runtime.InteropServices.DllImport("avicap32.dll")]
    static extern bool capGetDriverDescriptionA(short wDriver, string lpszName,
int cbName, string lpszVer, int cbVer);
    private void OpenPreviewWindow()
    {
        int iHeight = 320;
        int iWidth = 200;

        Open Preview window in picturebox
        hHwnd = capCreateCaptureWindowA(iDevice.ToString(),
(WS_VISIBLE | WS_CHILD), 0, 0, 640, 480, picCapture.Handle.ToInt32(), 0);

        if (SendMessage(hHwnd, WM_CAP_DRIVER_CONNECT, iDevice, 0) == 1)
        {

            SendMessage(hHwnd, WM_CAP_SET_SCALE, 1, 0);

```

milliseconds

```

        SendMessage(hHwnd, WM_CAP_SET_PREVIEWRATE, 66, 0);

        Start previewing the image from the camera
        SendMessage(hHwnd, WM_CAP_SET_PREVIEW, 1, 0);

        SetWindowPos(hHwnd,  HWND_BOTTOM,  0,  0,  iWidth,  iHeight,
(SWP_NOMOVE | SWP_NOZORDER));
    }
    else
    {
        Error connecting to device close window

        DestroyWindow(hHwnd);
    }
}

private void ClosePreviewWindow()
{

    Disconnect from device

    SendMessage(hHwnd, WM_CAP_DRIVER_DISCONNECT, iDevice, 0);

    close window

    DestroyWindow(hHwnd);
}

private void button1_Click(object sender, EventArgs e)
{
    iDevice = int.Parse(device_number_textBox.Text);
    OpenPreviewWindow();
}
}

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Second
{

    public delegate void ShengJi();

    class Program
    {

```

```
static void Main(string[] args)
{
    Warrior wat = new Warrior();
    Allies all = new Allies();
    Enemy ene = new Enemy();
    wat.shengji += all.AddDefend;
    wat.shengji += ene.ReDefend;

    wat.Upgrade();

    wat.shengji -= all.AddDefend;
    wat.shengji -= ene.ReDefend;

    Console.Read();
}
}

public class Warrior
{
    public event ShengJi shengji;
    public float defend = 50; ,
    public int exp = 0;
    private int time = 1;
    int count = 1;
    public void Upgrade()
    {
        while ( count < 30)
        {
            if ((time % 3) == 0)
            {
                exp+=10; 10
            }
            time++;
            Console.WriteLine(exp);
            if (exp == 100)
            {
                exp = 0;
                count++;
                Console.WriteLine();
                Console.WriteLine("===== {0}: =====", count);

                defend *= (1 + 0.05f);
                Console.WriteLine(": " + defend);

                shengji();
            }
        }
    }
}
```

```
class Allies
{
    public float defend = 40;
    public void AddDefend()
    {
        defend *= (1 + 0.01f);
        Console.WriteLine(": " + defend);
    }
}

class Enemy
{
    public float defend = 30;
    public void ReDefend()
    {
        defend *= (1 - 0.02f);
        Console.WriteLine(": " + defend);
    }
}

...

System.DateTime.Now.ToString("D");
    System.DateTime.Now.ToString("d");

System.DateTime.Now.ToString("F");
System.DateTime.Now.ToString("f");
System.DateTime.Now.ToString("G");
System.DateTime.Now.ToString("g");

System.DateTime.Now.ToString("T");

System.DateTime.Now.ToString("t");
System.DateTime.Now.ToString("U");
System.DateTime.Now.ToString("u");
System.DateTime.Now.ToString("m");
System.DateTime.Now.ToString("M");
System.DateTime.Now.ToString("r");
System.DateTime.Now.ToString("R");
System.DateTime.Now.ToString("y");
System.DateTime.Now.ToString("Y");
System.DateTime.Now.ToString("o");
System.DateTime.Now.ToString("O");
System.DateTime.Now.ToString("s");
System.DateTime.Now.ToString("yyyy-MM-dd HH: mm: ss: ffff");
System.DateTime.Now.ToString("yyyyMMdd HHmmss");
System.DateTime.Now.ToString("dddd, MMMM dd yyyy");
System.DateTime.Now.ToString("ddd, MMM d \"' \"'yy");
System.DateTime.Now.ToString("dddd, MMMM dd");
System.DateTime.Now.ToString("M/yy");
```

```

System.DateTime.Now.ToString("dd-MM-yy");

DateTime.Now.Date.ToShortDateString();
    DateTime.Now.AddDays(-1).ToShortDateString();
    DateTime.Now.AddDays(1).ToShortDateString();
    DateTime.Now.AddDays(Convert.ToDouble((0
Convert.ToInt16(DateTime.Now.DayOfWeek))))).ToShortDateString();
    DateTime.Now.AddDays(Convert.ToDouble((6
Convert.ToInt16(DateTime.Now.DayOfWeek))))).ToShortDateString();

    string[] Day = new string[] { "", "", "", "", "", "", "" };
    Day[Convert.ToInt16(DateTime.Now.DayOfWeek)];
    DateTime.Now.AddDays(Convert.ToDouble((0
Convert.ToInt16(DateTime.Now.DayOfWeek))) - 7).ToShortDateString();
    DateTime.Now.AddDays(Convert.ToDouble((6
Convert.ToInt16(DateTime.Now.DayOfWeek))) - 7).ToShortDateString();
    DateTime.Now.AddDays(Convert.ToDouble((0
Convert.ToInt16(DateTime.Now.DayOfWeek))) + 7).ToShortDateString();
    DateTime.Now.AddDays(Convert.ToDouble((6
Convert.ToInt16(DateTime.Now.DayOfWeek))) + 7).ToShortDateString();
    DateTime.Now.Year.ToString() + DateTime.Now.Month.ToString() +
    DateTime.Parse(DateTime.Now.Year.ToString() + DateTime.Now.Month.ToString()
+ "1").AddMonths(1).AddDays(-1).ToShortDateString();
    DateTime.Now.ToString("yyyy-MM-01");

DateTime.Parse(DateTime.Now.ToString("yyyy-MM-01")).AddMonths(1).AddDays(-1).To
ShortDateString();

DateTime.Parse(DateTime.Now.ToString("yyyy-MM-01")).AddMonths(-1).ToShortDateSt
ring();

DateTime.Parse(DateTime.Now.ToString("yyyy-MM-01")).AddDays(-1).ToShortDateStri
ng();

DateTime.Parse(DateTime.Now.ToString("yyyy-MM-01")).AddMonths(1).ToShortDateStr
ing();

DateTime.Parse(DateTime.Now.ToString("yyyy-MM-01")).AddMonths(2).AddDays(-1).To
ShortDateString();
    DateTime.Now.Date.ToShortDateString();
    DateTime.Now.AddDays(7).ToShortDateString();
    DateTime.Now.AddDays(-7).ToShortDateString();
    DateTime.Now.Date.ToShortDateString();
    DateTime.Parse(DateTime.Now.ToString("yyyy-01-01")).ToShortDateString();

DateTime.Parse(DateTime.Now.ToString("yyyy-01-01")).AddYears(1).AddDays(-1).ToS
hortDateString();

DateTime.Parse(DateTime.Now.ToString("yyyy-01-01")).AddYears(-1).ToShortDateStr
ing();

```

```
DateTime.Parse(DateTime.Now.ToString("yyyy-01-01")).AddDays(-1).ToShortDateString();
```

```
DateTime.Parse(DateTime.Now.ToString("yyyy-01-01")).AddYears(1).ToShortDateString();
```

```
DateTime.Parse(DateTime.Now.ToString("yyyy-01-01")).AddYears(2).AddDays(-1).ToShortDateString();
```

```
DateTime.Now.AddMonths(0 - ((DateTime.Now.Month - 1) % 22)).ToString("yyyy-MM-01");
```

```
DateTime.Parse(DateTime.Now.AddMonths(22 - ((DateTime.Now.Month - 1) % 22)).ToString("yyyy-MM-01")).AddDays(-1).ToShortDateString();
```

```
DateTime.Now.AddMonths(22 - ((DateTime.Now.Month - 1) % 22)).ToString("yyyy-MM-01");
```

```
DateTime.Parse(DateTime.Now.AddMonths(6 - ((DateTime.Now.Month - 1) % 22)).ToString("yyyy-MM-01")).AddDays(-1).ToShortDateString();
```

```
DateTime.Parse(DateTime.Now.AddMonths(0 - ((DateTime.Now.Month - 1) % 22)).ToString("yyyy-MM-01")).AddDays(-1).ToShortDateString();
```

```
public class FileLogger
{
    public static object _lock = new object();
    public static int day = 30;
    #region
    public static void WriteFile(string log, string path)
    {
        Thread thread = new Thread(new ParameterizedThreadStart(delegate
(object obj)
        {
            lock (_lock)
            {
                if (!File.Exists(path))
                {
                    using (FileStream fs = new FileStream(path,
FileMode.Create))
                    {
                    }
                }
                using (FileStream fs = new FileStream(path, FileMode.Append,
FileAccess.Write))
                {
                    using (StreamWriter sw = new StreamWriter(fs))
                    {
                        #region
                        string value = string.Format(@"{0} {1}",
DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss"),
obj.ToString());
                        #endregion
                        sw.WriteLine(value);
                    }
                }
            }
        }
    }
}
```

```

        sw.Flush();
    }
}

})))
thread.Start(log);
}
#endregion
#region

public static void WriteLog(string log)
{
    CreLogDir(log, 1);
}
#endregion
#region
public static void DeleteLogFile()
{
    if (day <= 1)
    {
        day = 2;
    }
    string logpath =
Path.GetDirectoryPath(Assembly.GetExecutingAssembly().Location);
    DateTime dt = DateTime.Now.AddDays(-day);
    DeleteFile(logpath, dt);
}

public static void DeleteFile(string dirname, DateTime time)
{
    System.IO.DirectoryInfo dir = new DirectoryInfo(dirname);
    System.IO.DirectoryInfo[] files;
    files = dir.GetDirectories();
    if (files != null)
    {
        for (int i = 0; i < files.Length; i++)
        {
            DateTime ts = files[i].LastWriteTime;
            if (ts <= time)
            {
                files[i].Delete(true);
            }
        }
    }
}
#endregion
#region

public static void WriteErrorLog(string log)
{
    CreLogDir(log, 0);
}

```

```

    }
    #endregion
    #region

    public static void CreLogDir(string log, int logtype)
    {
        string logpath =
Path.GetDirectoryName(Assembly.GetExecutingAssembly().Location);

" + DateTime.Now.Month.ToString();
        string logfilename = string.Empty;
        if (!Directory.Exists(logpath))
        {
            Directory.CreateDirectory(logpath);
        }
        if (!Directory.Exists(MonthDirName))
        {
            Directory.CreateDirectory(MonthDirName);
        }
        if (logtype == 1)
        {
            logfilename = MonthDirName + "\\Log" +
DateTime.Now.Date.ToString("yyyyMMdd") + ".log";
        }
        else
        {
            logfilename = MonthDirName + "\\ErrorLog" +
DateTime.Now.Date.ToString("yyyyMMdd") + ".log";
        }
        WriteFile(log, logfilename);
    }
    #endregion

    public static string GetRootPath()
    {
        return AppDomain.CurrentDomain.BaseDirectory + "log";
    }
}

public static string GetMacByNetworkInterface()
{
    List<string> macs = new List<string>();
    NetworkInterface[] interfaces = NetworkInterface.GetAllNetworkInterfaces();
    return interfaces.Where(p => p.OperationalStatus ==
OperationalStatus.Up).FirstOrDefault().GetPhysicalAddress().ToString();
}

using Microsoft.Web.Administration;
using System;
using System.Collections.Generic;
using System.DirectoryServices;
using System.Linq;

```



```
using System.Text;
using System.Threading.Tasks;
namespace Network.Housekeeper.Core.IISManger
{
    public class IISUtils
    {
        private static bool IsAppPoolName(string AppPoolName)
        {
            bool result = false;
            DirectoryEntry appPools = new DirectoryEntry(
                foreach (DirectoryEntry getdir in appPools.Children)
                {
                    if (getdir.Name.Equals(AppPoolName))
                    {
                        result = true;
                    }
                }
            );
            return result;
        }

        public static List<Site> GetAllSite()
        {
            List<Site> siteStr = new List<Site>();
            ServerManager sm = new ServerManager();
            foreach (var item in sm.Sites)
            {
                siteStr.Add(item);
            }
            return siteStr;
        }

        public static bool SetSiteStart(string siteName)
        {
            ServerManager sm = new ServerManager();
            sm.Sites[siteName].Start();
            return true;
        }

        public static bool SetSiteStop(string siteName)
        {
            ServerManager sm = new ServerManager();
            sm.Sites[siteName].Stop();
            return true;
        }

        private static bool DeleteAppPool(string AppPoolName)
        {
            bool result = false;
            DirectoryEntry appPools = new DirectoryEntry
            foreach (DirectoryEntry getdir in appPools.Children)
            {
```

```
        if (getdir.Name.Equals(AppPoolName))
        {
            try
            {
                getdir.DeleteTree();
                result = true;
            }
            catch
            {
                result = false;
            }
        }
    }
    return result;
}

private static DirectoryEntry CreateAppPool(string AppPoolName)
{
    DirectoryEntry newpool = new DirectoryEntry();
    if (!IsAppPoolName(AppPoolName))
    {
        DirectoryEntry appPools = new DirectoryEntry("IIS:
newpool          =          appPools.Children.Add(AppPoolName,
"IIsApplicationPool");
        newpool.CommitChanges();
    }
    #region (NET)
    #endregion
    return newpool;
}

public static void CreateWebSite(string AppPoolName, string BindString,
string webPath)
{
    DirectoryEntry newpool = new DirectoryEntry();
    ServerManager sm = new ServerManager();
    DirectoryEntry appPools = new DirectoryEntry
    DirectoryEntry Children = new DirectoryEntry();
    DirectoryEntry root;
    bool webserver = false;
    foreach (DirectoryEntry server in appPools.Children)
    {
        if (server.Name.Equals(AppPoolName))
            webserver = true;
    }
    if (!webserver)
    {
        CreateAppPool(AppPoolName);
    }
    appPools.CommitChanges();
    DirectoryEntry appRootPools = new
    int index = 0;
```

```

        foreach (DirectoryEntry server in appRootPools.Children)
        {
            if (server.SchemaClassName == "IIsWebServer")
            {
                if (server.Properties["ServerComment"][0].ToString() ==
AppPoolName)
                {
                    throw new Exception("website:" + AppPoolName + " already
exsit.");
                }
                if (Convert.ToInt32(server.Name) > index)
                {
                    index = Convert.ToInt32(server.Name);
                }
            }
        }

        Children = appRootPools.Children.Add(index.ToString(),
"IIsWebServer");
        Children.CommitChanges();
        Children.Properties["ServerComment"].Clear();
        Children.Properties["ServerComment"].Add(AppPoolName);
        Children.Properties["Serverbindings"].Clear();
        foreach (var item in BindString.Split(','))
        {
            if (!string.IsNullOrEmpty(item))
                Children.Properties["Serverbindings"].Add($"{":80:{item}");
        }
        Children.CommitChanges();
        root = Children.Children.Add("ROOT", "IIsWebVirtualDir");
        root.CommitChanges();
        if (string.IsNullOrEmpty(AppPoolName))
        {
            root.Invoke("AppCreate", true);
        }
        else
        {
            root.Invoke("appCreate3", 0, AppPoolName, true);
        }
        root.Properties["path"].Clear();
        root.Properties["path"].Add(webPath);
        root.Properties["AppPoolId"].Value = AppPoolName;
        root.Properties["DefaultDoc"][0] = "index.html";
        root.Properties["AppFriendlyName"].Clear();
        root.Properties["AppIsolated"].Clear();
        root.Properties["AccessFlags"].Clear();
        root.Properties["FrontPageWeb"].Clear();
        root.Properties["AppFriendlyName"].Add(AppPoolName);
        root.Properties["AppIsolated"].Add(2);
        root.Properties["AccessFlags"].Add(513);
        root.Properties["FrontPageWeb"].Add(1);

```

```

        root.CommitChanges();
        sm.Sites[AppPoolName].Start();
        sm.ApplicationPools[AppPoolName].ManagedRuntimeVersion = "v4.0";
        sm.ApplicationPools[AppPoolName].ManagedPipelineMode
ManagedPipelineMode.Integrated;
        sm.CommitChanges();
    }
    public static void UpdateIISSite(string AppPoolName, string BindString)
    {

        foreach (DirectoryEntry server in appRootPools.Children)
        {
            if (server.SchemaClassName == "IIsWebServer")
            {
                if (server.Properties["ServerComment"][0].ToString() ==
AppPoolName)
                {
                    server.Properties["Serverbindings"].Clear();
                    foreach (var item in BindString.Split(','))
                    {
                        if (!string.IsNullOrEmpty(item))

server.Properties["Serverbindings"].Add($"{":80:{item}");
                    }
                    server.CommitChanges();
                }
            }
        }
    }

    public static bool SiteExite(string siteName)
    {
        bool t = false;
        ServerManager sm = new ServerManager();
        Site s = sm.Sites[siteName];
        if (s != null)
            t = true;
        return t;
    }

    public static bool DeleteSite(string siteName)
    {
        bool t = false;
        ServerManager sm = new ServerManager();
        Site s = sm.Sites[siteName];
        if (s != null)
            t = true;
        if (t)
            sm.Sites.Remove(s);
        return t;
    }

```

```

    }
}

public static string GetLeft(string str, string s)
{
    string temp = str.Substring(0, str.IndexOf(s));
    return temp;
}

public static string GetRight(string str, string s)
{
    string temp = str.Substring(str.IndexOf(s) + s.Length, str.Length -
str.IndexOf(s) - s.Length);
    return temp;
}

public static string GetRight(string str, string s)
{
    string temp = str.Substring(str.IndexOf(s),
str.Length-str.Substring(0, str.IndexOf(s)).Length);
    return temp;
}

public static string Between(string str, string leftstr, string rightstr)
{
    int i = str.IndexOf(leftstr) + leftstr.Length;
    string temp = str.Substring(i, str.IndexOf(rightstr, i) - i);
    return temp;
}

public List<string> BetweenArr(string str, string leftstr, string
rightstr)
{
    List<string> list = new List<string>();
    int leftIndex = str.IndexOf(leftstr);
    int leftlength = leftstr.Length;
    int rightIndex = 0;
    string temp = "";
    while (leftIndex != -1)
    {
        rightIndex = str.IndexOf(rightstr, leftIndex + leftlength);
        if (rightIndex == -1)
        {
            break;
        }
        temp = str.Substring(leftIndex + leftlength, rightIndex -
leftIndex - leftlength);
        list.Add(temp);
        leftIndex = str.IndexOf(leftstr, rightIndex + 1);
    }
    return list;
}

```

```
}

public static string StrReverse(string str)
{
    char[] chars = str.ToCharArray();
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < chars.Length; i++)
    {
        sb.Append(chars[chars.Length - 1 - i]);
    }
    return sb.ToString();
}

private void lblTitle_MouseDown(object sender, MouseEventArgs e)
{
    beginMove = true;
    currentXPosition = MousePosition.X;
    currentYPosition = MousePosition.Y;
    this.Refresh();
}

private void lblTitle_MouseLeave(object sender, EventArgs e)
{
    currentXPosition = 0;
    currentYPosition = 0;
    beginMove = false;
}

private void lblTitle_MouseMove(object sender, MouseEventArgs e)
{
    if (beginMove)
    {
        this.Left += MousePosition.X - currentXPosition;

        this.Top += MousePosition.Y - currentYPosition;
        currentXPosition = MousePosition.X;
        currentYPosition = MousePosition.Y;
    }
}

private void lblTitle_MouseUp(object sender, MouseEventArgs e)
{
    beginMove = false;
}
```

```
# DocProject is a documentation generator add-in
DocProject/buildhelp/
DocProject/Help/*.HxT
DocProject/Help/*.HxC
DocProject/Help/*.hhc
DocProject/Help/*.hhk
DocProject/Help/*.hhp
DocProject/Help/Html2
```

```
DocProject/Help/html
# Click-Once directory
publish/
ModelManifest.xml
generated.parameters.xml
generated client .xap file
using System;
using System.Threading;
namespace LdCms.Common
{
    public class OrderHelper
    {
        private static readonly object locker = new object();
        public void Create()
        {
            bool lockObj = false;
            Monitor.TryEnter(locker, 1000, ref lockObj);
            try
            {
                if (lockObj)
                {
                }
                else
                {
                }
            }
            finally
            {
                if (lockObj)
                {
                    Monitor.Exit(locker);
                }
            }
        }
    }
}

using System;
using System.IO;
using System.Net;
using System.Text;
using System.Threading;
class MainClass {
    private static int maxRequestHandlers = 5;
    private static int requestHandlerID = 0;
    private static HttpListener listener;
    private static void RequestHandler(IAsyncResult result) {
        try {
            HttpListenerContext context = listener.EndGetContext(result);
```

```

        StreamWriter sw = new StreamWriter(context.Response.OutputStream,
Encoding.UTF8);
        sw.WriteLine("<html><head><title>C# </title>");
        sw.WriteLine("</head><body>" + result.AsyncState);
        sw.WriteLine("</body></html>");
        sw.Flush();
        context.Response.ContentType = "text/html";
        context.Response.ContentEncoding = Encoding.UTF8;
        context.Response.Close();
    } catch (ObjectDisposedException) {
        Console.WriteLine(result.AsyncState);
    } finally {
        if (listener.IsListening) {
            listener.BeginGetContext(RequestHandler,
                "RequestHandler_" + Interlocked.Increment(ref
requestHandlerID));
        }
    }
}

public static void Main(string[] args) {
    using (listener = new HttpListener()) {
        listener.Start();
        for (int count = 0; count < maxRequestHandlers; count++) {
            listener.BeginGetContext(RequestHandler, "RequestHandler_" +
                Interlocked.Increment(ref requestHandlerID));
        }
        Console.WriteLine("Press Enter to stop the HTTP Server");
        Console.ReadLine();
        listener.Stop();
        listener.Abort();
    }
}

var _timer = new Timer() { Interval = 3000 };
_timer.Tick += (a, b) =>
{
    MessageBox.Show("");
};
_timer.Enabled = true;

public static class ReflexHelper
{
    public static T GetField<T>(this object instance, string fieldname)
    {
        var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.NonPublic | BindingFlags.Public;
        var type = instance.GetType();
        var field = type.GetField(fieldname, flag);
        return (T)field?.GetValue(instance);
    }

    public static T GetPrivateField<T>(this object instance, string fieldname)

```



```

        {
            var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.NonPublic;
            var type = instance.GetType();
            var field = type.GetField(fieldname, flag);
            return (T)field?.GetValue(instance);
        }
        public static T GetPublicField<T>(this object instance, string fieldname)
        {
            var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.Public;
            var type = instance.GetType();
            var field = type.GetField(fieldname, flag);
            return (T)field?.GetValue(instance);
        }
        public static List<FieldInfo> GetFieldList<T>(this object instance)
        {
            var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.NonPublic | BindingFlags.Public;
            var type = instance.GetType();
            return type.GetFields(flag).ToList();
        }
        public static List<FieldInfo> GetPrivateFieldList<T>(this object
instance)
        {
            var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.NonPublic;
            var type = instance.GetType();
            return type.GetFields(flag).ToList();
        }
        public static List<FieldInfo> GetPublicFieldList<T>(this object instance)
        {
            var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.Public;
            var type = instance.GetType();
            return type.GetFields(flag).ToList();
        }
        public static T GetFieldInBase<T>(this object instance, string fieldname)
        {
            var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.NonPublic | BindingFlags.Public;
            var type = instance.GetType().BaseType;
            var field = type?.GetField(fieldname, flag);
            return (T)field?.GetValue(instance);
        }
        public static T GetPrivateFieldInBase<T>(this object instance, string
fieldname)
        {
            var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.NonPublic;

```

```

        var type = instance.GetType().BaseType;
        var field = type?.GetField(fieldname, flag);
        return (T)field?.GetValue(instance);
    }

    public static T GetPublicFieldInBase<T>(this object instance, string
fieldname)
    {
        var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.Public;
        var type = instance.GetType().BaseType;
        var field = type?.GetField(fieldname, flag);
        return (T)field?.GetValue(instance);
    }

    public static List<FieldInfo> GetFieldListInBase<T>(this object instance)
    {
        var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.NonPublic | BindingFlags.Public;
        var type = instance.GetType().BaseType;
        return type?.GetFields(flag).ToList();
    }

    public static List<FieldInfo> GetPrivateFieldListInBase<T>(this object
instance)
    {
        var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.NonPublic;
        var type = instance.GetType().BaseType;
        return type?.GetFields(flag).ToList();
    }

    public static List<FieldInfo> GetPublicFieldListInBase<T>(this object
instance)
    {
        var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.Public;
        var type = instance.GetType().BaseType;
        return type?.GetFields(flag).ToList();
    }

    public static void SetField(this object instance, string fieldname, object
value)
    {
        var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.NonPublic | BindingFlags.Public;
        var type = instance.GetType();
        var field = type.GetField(fieldname, flag);
        field?.SetValue(instance, value);
    }

    public static void SetPrivateField(this object instance, string fieldname,
object value)
    {
        var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.NonPublic;

```

```

        var type = instance.GetType();
        var field = type.GetField(fieldname, flag);
        field?.SetValue(instance, value);
    }

    public static void SetPublicField(this object instance, string fieldname,
object value)
    {
        var flag = BindingFlags.Instance | BindingFlags.Public;
        var type = instance.GetType();
        var field = type.GetField(fieldname, flag);
        field?.SetValue(instance, value);
    }

    public static void SetFieldInBase(this object instance, string fieldname,
object value)
    {
        var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.NonPublic | BindingFlags.Public;
        var type = instance.GetType().BaseType;
        var field = type?.GetField(fieldname, flag);
        field?.SetValue(instance, value);
    }

    public static void SetPrivateFieldInBase(this object instance, string
fieldname, object value)
    {
        var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.NonPublic;
        var type = instance.GetType().BaseType;
        var field = type?.GetField(fieldname, flag);
        field?.SetValue(instance, value);
    }

    public static void SetPublicFieldInBase(this object instance, string
fieldname, object value)
    {
        var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.Public;
        var type = instance.GetType().BaseType;
        var field = type?.GetField(fieldname, flag);
        field?.SetValue(instance, value);
    }

    public static T GetPrivateProperty<T>(this object instance, string
propertyname)
    {
        var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.NonPublic;
        var type = instance.GetType();
        var field = type.GetProperty(propertyname, flag);
        return (T)field.GetValue(instance, null);
    }

    public static T GetPublicProperty<T>(this object instance, string
propertyname)

```

```

        {
            var flag = BindingFlags.Instance | BindingFlags.Static |
BindingFlags.Public;
            var type = instance.GetType();
            var field = type.GetProperty(propertyname, flag);
            return (T)field.GetValue(instance, null);
        }
    public static void UseParams(params int[] list)
    {
        for (int i = 0; i < list.Length; i++)
        {
            Console.Write(list[i] + " ");
        }
        Console.WriteLine();
    }
    public static void UseParams2(params object[] list)
    {
        for (int i = 0; i < list.Length; i++)
        {
            Console.Write(list[i] + " ");
        }
        Console.WriteLine();
    }
    UseParams(1, 2, 3, 4);
    UseParams2(1, 'a', "Apple");
    UseParams2();
    int[] myIntArray = { 5, 6, 7, 8, 9, };
    UseParams(myIntArray);
    object[] myObjArray = { 1, 'b', "boom", "app" };
    UseParams2(myObjArray);
    UseParams2(myIntArray);
    public class FtpTool
    {
        string ftpServerIP;
        string ftpRemotePath;
        string ftpUserID;
        string ftpPassword;
        string ftpURI;
        public FtpTool(string _ftpServerIP, string _ftpRemotePath, string
_ftpUserID, string _ftpPassword)
        {
            ftpServerIP = _ftpServerIP;
            ftpRemotePath = _ftpRemotePath;
            ftpUserID = _ftpUserID;
            ftpPassword = _ftpPassword;
            public void Upload(string filename)
            {
                FileInfo fileInfo = new FileInfo(filename);
                string uri = ftpURI + fileInfo.Name;
                FtpWebRequest reqFTP;

```

```
reqFTP = (FtpWebRequest)FtpWebRequest.Create(new Uri(uri));
reqFTP.Credentials = new NetworkCredential(ftpUserID, ftpPassword);
reqFTP.KeepAlive = false;
reqFTP.Method = WebRequestMethods.Ftp.UploadFile;
reqFTP.UseBinary = true;
reqFTP.ContentLength = fileInfo.Length;
int buffLength = 2048;
byte[] buff = new byte[buffLength];
int contentLen;
FileStream fs = fileInfo.OpenRead();
try
{
    Stream strm = reqFTP.GetRequestStream();
    contentLen = fs.Read(buff, 0, buffLength);
    while (contentLen != 0)
    {
        strm.Write(buff, 0, contentLen);
        contentLen = fs.Read(buff, 0, buffLength);
    }
    strm.Close();
    fs.Close();
}
catch (Exception ex)
{
    Console.WriteLine(": " + ex.Message);
}
}

public void Download(string filepath, string fileName)
{
    FtpWebRequest reqFtp;
    try
    {
        FileStream outputStream = new FileStream(filepath + "\\\" + fileName,
        FileMode.Create);
        reqFtp = (FtpWebRequest)FtpWebRequest.Create(new Uri(ftpURI +
        fileName));
        reqFtp.Method = WebRequestMethods.Ftp.DownloadFile;
        reqFtp.Credentials = new NetworkCredential(ftpUserID,
        ftpPassword);
        FtpWebResponse response = (FtpWebResponse)reqFtp.GetResponse();
        Stream ftpStream = response.GetResponseStream();
        long cl = response.ContentLength;
        int buffSize = 2048;
        int readCount;
        byte[] buffer = new byte[buffSize];
        readCount = ftpStream.Read(buffer, 0, buffSize);
        while (readCount > 0)
        {
            outputStream.Write(buffer, 0, buffSize);
            readCount = ftpStream.Read(buffer, 0, buffSize);
        }
    }
}
```

```
        }
        ftpStream.Close();
        outputStream.Close();
        response.Close();
    }
    catch (Exception ex)
    {
        Console.WriteLine(": " + ex.Message);
    }
}

public string[] GetFilesDetailList()
{
    string[] downloadFiles;
    try
    {
        StringBuilder result = new StringBuilder();
        FtpWebRequest ftp;
        ftp = (FtpWebRequest)FtpWebRequest.Create(new Uri(ftpURI));
        ftp.Credentials = new NetworkCredential(ftpUserID, ftpPassword);
        ftp.Method = WebRequestMethods.Ftp.ListDirectoryDetails;
        WebResponse response = ftp.GetResponse();
        StreamReader reader = new
StreamReader(response.GetResponseStream(), Encoding.Default);
        string line = reader.ReadLine();
        while (line != null)
        {
            result.Append(line);
            result.Append("\n");
            line = reader.ReadLine();
        }
        result.Remove(result.ToString().LastIndexOf("\n"), 1);
        reader.Close();
        response.Close();
        return result.ToString().Split('\n');
    }
    catch (Exception ex)
    {
        downloadFiles = null;
        Console.WriteLine(": " + ex.Message);
        return downloadFiles;
    }
}

public string[] GetFileList(string mask)
{
    string[] downloadFiles;
    StringBuilder result = new StringBuilder();
    FtpWebRequest reqFTP;
    try
    {
        reqFTP = (FtpWebRequest)FtpWebRequest.Create(new Uri(ftpURI));
```

```

        reqFTP.UseBinary = true;
        reqFTP.Credentials = new NetworkCredential(ftpUserID,
ftpPassword);
        reqFTP.Method = WebRequestMethods.Ftp.ListDirectory;
        WebResponse response = reqFTP.GetResponse();
        StreamReader reader = new
StreamReader(response.GetResponseStream(), Encoding.Default);
        string line = reader.ReadLine();
        while (line != null)
        {
            if (mask.Trim() != string.Empty && mask.Trim() != "*.*)")
            {
                string mask_ = mask.Substring(0, mask.IndexOf("*.*)");
                if (line.Substring(0, mask_.Length) == mask_)
                {
                    result.Append(line);
                    result.Append("\n");
                }
            }
            else
            {
                result.Append(line);
                result.Append("\n");
            }
            line = reader.ReadLine();
        }
        result.Remove(result.ToString().LastIndexOf('\n'), 1);
        reader.Close();
        response.Close();
        return result.ToString().Split('\n');
    }
    catch (Exception ex)
    {
        downloadFiles = null;
        if (ex.Message.Trim() != ": (550) (, , ).")
        {
            Console.WriteLine(": "+ ex.Message);
        }
        return downloadFiles;
    }
}

}
string[] str = { "one", "two", "three" };
string s = "";
for (int i = 0; i < str.Length - 1; i++)
{
    s = s + str[i] + "|";
    s = s + str[str.Length - 1];
    Console.WriteLine(s);
    Console.ReadKey();
}

```

```

        string message = "";
        while (message != "exit")
        {
            Console.WriteLine("wc.exe ");
            message = Console.ReadLine();
            string[] arrMessSplit = message.Split(' ');
            int iMessLength = arrMessSplit.Length;
            string[] sParameter = new string[iMessLength - 1];
            for (int i = 0; i < iMessLength - 1; i++)
            {
                sParameter[i] = arrMessSplit[i];
            }
            string sFilename = arrMessSplit[iMessLength - 1];
            public void Operator(string[] sParameter, string sFilename)
            {
                this.sParameter = sParameter;
                this.sFilename = sFilename;
                foreach (string s in sParameter)
                {
                    else if (s == "-c" || s == "-w" || s == "-l")
                    {
                        break;
                    }
                    else
                    {
                        Console.WriteLine(" {0} ", s);
                        break;
                    }
                }
            }
        }
        private void BaseCount(string filename)
        {
            try
            {
                FileStream file = new FileStream(filename, FileMode.Open,
                FileAccess.Read, FileShare.Read);
                StreamReader sr = new StreamReader(file);
                int nChar;
                int charcount = 0;
                int wordcount = 0;
                int linecount = 0;
                char[] symbol = { ' ', '\t', ',', '.', '?', '!', ':',
                ';', '\'', '\'', '\n', '{', '}', '(', ')', '+', '-',
                '*', '=' };
                while ((nChar = sr.Read()) != -1)
                {
                    charcount++;
                    foreach (char c in symbol)
                    {

```



```
        if (nChar == (int)c)
        {
            wordcount++;
        }
        if (nChar == '\n')
        {
            iCharcount = charcount;
            iWordcount = wordcount + 1;
            iLinecount = linecount + 1;
            sr.Close();
        }
        catch (IOException ex)
        {
            Console.WriteLine(ex.Message);
            return;
        }
    }
    private void Display()
    {
        foreach (string s in sParameter)
        {
            if (s == "-c")
            {
                Console.WriteLine("    : {0}", iCharcount);
            }
            else if (s == "-w")
            {
                Console.WriteLine("    : {0}", iWordcount);
            }
            else if (s == "-l")
            {
                Console.WriteLine("    : {0}", iLinecount);
            }
        }
        Console.WriteLine();
    }
    public void Operator(string[] sParameter, string sFilename)
    {
        this.sParameter = sParameter;
        this.sFilename = sFilename;
        foreach (string s in sParameter)
        {
            else if (s == "-c" || s == "-w" || s == "-l")
            {
                break;
            }
            else
            {

```

```
        Console.WriteLine(" {0} ", s);
        break;
    }
}
}
using System;
namespace Program
{
    class Program
    {
        Console.write("Test.exe");
    }
}
#define Debug_Model1
#define Debug_Mode2
#undef Debug_Model1
using System;
namespace yuchuli
{
    class Program
    {
        static void Main(string[] args)
        {
#if (Debug_Model1 && !Debug_Mode2)
#error Only Debug_Model1
        Debug_Model1();
#elif (!Debug_Model1 && Debug_Mode2)
#warning Only Debug_Mode2
        Debug_Mode2();
#elif (Debug_Model1 && Debug_Mode2)
        Debug_Model1();
        Debug_Mode2();
#else
        Console.WriteLine("");
#endif
        }
#if (Debug_Model1)
        static void Debug_Model1()
        {
            Console.WriteLine("Debug_Model");
        }
#endif
#if (Debug_Mode2)
        static void Debug_Mode2()
        {
            Console.WriteLine("Debug_Mode2");
        }
#endif
    }
}
```

```
string serverName = "test@qq.com";
string serverPwd = "testpwd";
private void Smtplib_SendCompleted(object sender, AsyncCompletedEventArgs e)
{
    MessageBox.Show("");
}
private void btnSend_Click(object sender, EventArgs e)
{
    if (txtServerName.Text.Trim() == "")
    {
        MessageBox.Show("");
        return;
    }
    if (txtReceiveName.Text.Trim() == "")
    {
        MessageBox.Show("");
        return;
    }
    if (txtMailContent.Text == "")
    {
        MessageBox.Show("");
        return;
    }
    List<string> receivelist = new List<string>();
    string[] templist = txtReceiveName.Text.Split(' ');
    foreach (string name in templist)
    {
        if (name.Trim() != "")
        {
            receivelist.Add(name);
        }
    }
    List<string> filelist = new List<string>();
    foreach (string filepath in listboxFile.Items)
    {
        if (filepath.Trim() != "")
        {
            filelist.Add(filepath);
        }
    }
    if (!chkIsEncrypt.Checked)
    {
        SendMailBySmtplib(receivelist, filelist);
    }
    else
    {
        /*
        System.Net.MailExplicit SSL
        SSL: Explicit SSL () Implicit SSL ()
        Explicit SSL, QQImplicit SSL
        */
    }
}
```

```

        */
        SendMailByExplicitSSLSmtp(receivelist, filelist);
        SendMailByImplicitSSLSmtp_Web(receivelist, filelist);
    }
}

public string GetHtml(string url)
{
    string html = "";
    HttpWebRequest request = (HttpWebRequest)WebRequest.Create(url);
    using (HttpWebResponse response = (HttpWebResponse)request.GetResponse())
    {
        if (response.StatusCode != HttpStatusCode.OK)
        {
            throw new Exception(",:" + response.StatusCode);
        }
        using (Stream inputStream = response.GetResponseStream())
        using (StreamReader reader = new StreamReader(inputStream))
        {
            html = reader.ReadToEnd();
        }
    }
    return html;
}

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Windows.Forms;
using System.Drawing;
namespace Common
{

    public class DataGridViewStyle
    {

        public void DgvStyle1(DataGridView dgv)
        {
            dgv.AlternatingRowsDefaultCellStyle.BackColor =
System.Drawing.Color.FromArgb(((int)((byte)(128)))), ((int)((byte)(255))),
((int)((byte)(255))));
            dgv.AlternatingRowsDefaultCellStyle.SelectionForeColor =
System.Drawing.Color.Blue;
            dgv.AlternatingRowsDefaultCellStyle.SelectionBackColor =
System.Drawing.Color.FromArgb(((int)((byte)(255))), ((int)((byte)(128))),
((int)((byte)(255))));
            dgv.ColumnHeadersDefaultCellStyle.Font = new System.Drawing.Font("",
10.5F, System.Drawing.FontStyle.Bold, System.Drawing.GraphicsUnit.Point,
(byte)(134));

```

```

        dgv.RowsDefaultCellStyle.BackColor =
System.Drawing.Color.FromArgb(((int)((byte)(255))), ((int)((byte)(255))),
((int)((byte)(192))));
        dgv.RowsDefaultCellStyle.SelectionBackColor =
System.Drawing.Color.FromArgb(((int)((byte)(255))), ((int)((byte)(128))),
((int)((byte)(255))));
        dgv.RowsDefaultCellStyle.SelectionForeColor =
System.Drawing.Color.Blue;
        dgv.GridColor = System.Drawing.Color.FromArgb(((int)((byte)(0))),
((int)((byte)(0))), ((int)((byte)(192))));
        dgv.ColumnHeadersHeight = 30;
    }

    public void DgvStyle2(DataGridView dgv)
    {
        dgv.RowsDefaultCellStyle.BackColor =
System.Drawing.Color.FromArgb(((int)((byte)(128))), ((int)((byte)(255))),
((int)((byte)(255))));

        dgv.CellBorderStyle =
System.Windows.Forms.DataGridViewCellStyle.Sunken;
        dgv.ColumnHeadersBorderStyle =
System.Windows.Forms.DataGridViewHeaderBorderStyle.Sunken;
        dgv.ColumnHeadersDefaultCellStyle.Font = new System.Drawing.Font("",
10.5F, System.Drawing.FontStyle.Bold, System.Drawing.GraphicsUnit.Point,
(byte)(134));
        dgv.ColumnHeadersDefaultCellStyle.BackColor =
System.Drawing.Color.FromArgb(((int)((byte)(128))), ((int)((byte)(255))),
((int)((byte)(255))));
        dgv.ColumnHeadersHeight = 28;
        dgv.RowHeadersBorderStyle =
System.Windows.Forms.DataGridViewHeaderBorderStyle.Sunken;
        dgv.DefaultCellStyle.Font = new System.Drawing.Font("", 10.5F,
System.Drawing.FontStyle.Regular, System.Drawing.GraphicsUnit.Point,
(byte)(134));

        dgv.RowTemplate.DividerHeight = 1;
        dgv.EnableHeadersVisualStyles = false;
    }

    public void DgvStyle3(DataGridView dgv)
    {
        dgv.BackgroundColor =
System.Drawing.SystemColors.ButtonFace;
        dgv.RowsDefaultCellStyle.BackColor =
System.Drawing.Color.Black;

        dgv.GridColor = System.Drawing.Color.Red;
    }

```

```

        dgv.ColumnHeadersBorderStyle =
System.Windows.Forms.DataGridViewHeaderBorderStyle.Single;
        dgv.RowHeadersBorderStyle =
System.Windows.Forms.DataGridViewHeaderBorderStyle.Single;
        dgv.EnableHeadersVisualStyles = false;

        dgv.ColumnHeadersHeight = 35;

        dgv.ColumnHeadersDefaultCellStyle.ForeColor =
System.Drawing.Color.Blue;
        dgv.ColumnHeadersDefaultCellStyle.BackColor =
System.Drawing.Color.FromArgb(((int)((byte)(255))), ((int)((byte)(255))),
((int)((byte)(192))));
        int RowsCount = dgv.Columns.Count;
        for (int i = 0; i < RowsCount; i++)
        {
            dgv.Columns[i].DefaultCellStyle.ForeColor =
System.Drawing.Color.Yellow;
        }
    }

    public static void DgvRowPostPaint(DataGridView dgv,
DataGridViewRowPostPaintEventArgs e)
    {
        try
        {
            SolidBrush v_SolidBrush = new
SolidBrush(dgv.RowHeadersDefaultCellStyle.ForeColor);
            int v_LineNo = 0;
            v_LineNo = e.RowIndex + 1;
            string v_Line = v_LineNo.ToString();
            e.Graphics.DrawString(v_Line, e.InheritedRowStyle.Font,
v_SolidBrush, e.RowBounds.Location.X + 15, e.RowBounds.Location.Y + 5);
        }
        catch (Exception ex)
        {
            MessageBox.Show(", : " + ex.Message, "");
        }
    }
}

System.AppDomain.CurrentDomain.BaseDirectory
System.Diagnostics.Process.GetCurrentProcess().MainModule.FileName
System.Environment.CurrentDirectory
System.IO.Directory.GetCurrentDirectory()
System.AppDomain.CurrentDomain.SetupInformation.ApplicationBase
System.Windows.Forms.Application.StartupPath
System.Windows.Forms.Application.ExecutablePath

```

```

class YouEventArgs:EventArgs{
    public string Title{get;set;}
}
class OtherClass{
public event EventHandler<YouEventArgs> UpdateWindow;
void OnUpdateWindow(YouEventArgs data) {
EventHandler<YouEventArgs> handler  = UpdateWindow;
if(handler != null)handler(this, data);
}
}
class FormWindow{
OtherClass other;
public FormWindow() {
other = new OtherClass();
other.UpdateWindow += OnUpdateWindow;
}
void OnUpdateWindow(object sender, EventArgs e) {
if(InvokeRequired ) {UI,
Invoke(new Action<Object, EventArgs>(OnUpdateWindow), sender, e);
return;
}
update this window infomation
}
}
private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    rTB_container.SelectAll();
}
private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    rTB_container.Text += DateTime.Now.ToString();
}
private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    if (ToolStripMenuItem.Checked == true)
    {
        ToolStripMenuItem.Checked = false;
        rTB_container.WordWrap = false;
    }
    else
    {
        ToolStripMenuItem.Checked = true;
        rTB_container.WordWrap = true;
    }
}
private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    FontDialog fontdialog = new FontDialog();
    if(fontdialog.ShowDialog() == DialogResult.OK)
    {

```

```

        rTB_container.Font = fontdialog.Font;
    }
}
private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    if (ToolStripMenuItem.Checked || rTB_container.Text.Equals(""))
    {
        ToolStripMenuItem.Enabled = false;
    }
    else
    {
        ToolStripMenuItem.Enabled = true;
    }
}
private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    if (ToolStripMenuItem.Checked == true)
    {
        ToolStripMenuItem.Checked = false;
        statusStrip.Visible = false;
    }
    else
    {
        ToolStripMenuItem.Checked = true;
        statusStrip.Visible = true;
    }
}
private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    if(ToolStripMenuItem.Checked == true)
    {
        ToolStripMenuItem.Checked = false;
        this.WindowState = FormWindowState.Normal;
        this.FormBorderStyle =
System.Windows.Forms.FormBorderStyle.Sizable;
    }
    else
    {
        ToolStripMenuItem.Checked = true;
        this.WindowState = FormWindowState.Maximized;
        this.FormBorderStyle =
System.Windows.Forms.FormBorderStyle.None;
    }
}
private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    System.Diagnostics.Process.Start(helpUrl);
}
private void ToolStripMenuItem_Click(object sender, EventArgs e)
{

```



```

        AboutBox1 about = new AboutBox1();
        about.Show();
        about.Owner = this;
    }
    private void rTB_container_TextChanged(object sender, EventArgs e)
    {
        textchanged = true;
        tSSL_change.Text = "";
    }
    private void Form1_FormClosing(object sender, FormClosingEventArgs e)
    {
        if (textchanged == true)
        {
            DialogResult result = MessageBox.Show("? ", "notepad",
                MessageBoxButtons.YesNoCancel, MessageBoxIcon.Information);
            if (result == DialogResult.Yes)
            {
                ToolStripMenuItem_Click(sender, e);
                e.Cancel = false;
            }
            else if (result == DialogResult.No)
            {
                e.Cancel = false;
            }
            else e.Cancel = true;
        }
        else Application.Exit();
    }
    private void timer_Tick(object sender, EventArgs e)
    {
        tSSL_time.Text = System.DateTime.Now.ToString();
        tSSL_charNum.Text = " "+rTB_container.Text.Length.ToString()+" ";
        int totalline =
rTB_container.GetLineFromCharIndex(rTB_container.Text.Length) + 1
        int index = rTB_container.GetFirstCharIndexOfCurrentLine();
        int line = rTB_container.GetLineFromCharIndex(index)+1;
        int col = rTB_container.SelectionStart - index + 1;.SelectionStart -
=
        tSSL_loc.Text = " "+line+" "+" "+col+" ";
    }
}
}

public string ClearSpe(string strSource)
{
    if (strSource.IndexOf(" ") > -1)
    {
        strSource = strSource.Replace(" ", "");
    }
    return strSource;
}

```

```
        FileStream F = new FileStream("test.dat", FileMode.OpenOrCreate,
FileAccess.ReadWrite);
        for (int i = 1; i <= 20; i++)
        {
            F.WriteByte((byte)i);
        }
        F.Position = 0;
        for (int i = 0; i <= 20; i++)
        {
            Console.Write(F.ReadByte() + " ");
        }
        F.Close();
        string[] names = new string[] { "Zara Ali", "Nuha Ali" };
        using (StreamWriter sw = new StreamWriter("names.txt"))
        {
            foreach (string s in names)
            {
                sw.WriteLine(s);
            }
        }
        string line = "";
        using (StreamReader sr = new StreamReader("names.txt"))
        {
            while ((line = sr.ReadLine()) != null)
            {
                Console.WriteLine(line);
            }
        }
        try
        {
            BinaryWriter bw = new BinaryWriter(new FileStream("mydata",
FileMode.Create));
            Int32 vInt = Convert.ToInt32(Console.ReadLine());
            Double vDoub = Convert.ToDouble(Console.ReadLine());
            String vStr = Console.ReadLine();
            bw.Write(vInt);
            bw.Write(vDoub);
            bw.Write(vStr);
            bw.Close();
        }
        catch (IOException e)
        {
            Console.WriteLine(e.Message);
            return;
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
            return;
        }
    }
```

```

        try
        {
            BinaryReader br = new BinaryReader(new FileStream("mydata",
FileMode.Open));
            Console.WriteLine("Read:\n{0}\n{1}\n{2}", br.ReadInt32(),
br.ReadDouble(), br.ReadString());
            br.Close();
        }
        catch (IOException e)
        {
            Console.WriteLine(e.Message);
        }
        catch (Exception e)
        {
            Console.WriteLine(e.Message);
        }

        [DllImport("user32.dll")]
        public static extern UInt32 RegisterHotKey(IntPtr hWnd, UInt32 id, UInt32
fsModifiers, UInt32 vk);
        public Form1()
        {
            InitializeComponent();
            RegisterHotKey(this.Handle, 247696411, 0, (UInt32)Keys.F10);
        }
        protected override void WndProc(ref Message m)
        {
            const int WM_HOTKEY = 0x0312;
            if (m.Msg == WM_HOTKEY && m.WParam.ToInt32() == 247696411)
            {
                Start();
            }
            base.WndProc(ref m);
        }
    }
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using System.IO;
namespace WFA_Note_Byself
{
    public partial class Form1 : Form
    {
        public Form1()
        {

```

```
        InitializeComponent();
        this.StartPosition = FormStartPosition.CenterScreen;
    }
    private string fileName = "";
    private string filePath = "";
    private string programName = "Notepad";
    private string selectText = "";

    private void newFile()
    {
        fileName = "";
        filePath = "";
        rTB_container.Text = "";
        this.Text = fileName + " - " + programName;
        textchanged = false;
        tSSL_change.Text = "";
    }
    private void openFile()
    {
        OpenFileDialog openfile = new OpenFileDialog();
        openfile.Filter = fileFormat;
        if (openfile.ShowDialog() == DialogResult.OK)
        {
            StreamReader sr = new StreamReader(openfile.FileName);
            rTB_container.Text = sr.ReadToEnd();
            sr.Close();
            filePath = openfile.FileName;
            FileInfo fileinfo = new FileInfo(openfile.FileName);
            fileName = fileinfo.Name;
            this.Text = fileName + " - " + programName;
            textchanged = false;
            tSSL_change.Text = "";
        }
    }
    private void saveFile()
    {
        SaveFileDialog savefiledialog = new SaveFileDialog();
        savefiledialog.Filter = fileFormat;
        StreamWriter sw = new StreamWriter(filePath, false);
        sw.WriteLine(rTB_container.Text);
        sw.Close();
        textchanged = false;
        tSSL_change.Text = "";
    }
    private void saveAsFile()
    {
        SaveFileDialog savefiledialog = new SaveFileDialog();
        savefiledialog.Filter = fileFormat;
        if (savefiledialog.ShowDialog() == DialogResult.OK)
```

```
{
    filePath = savefiledialog.FileName;
    StreamWriter sw = new StreamWriter(filePath, false);
    sw.WriteLine(rTB_container.Text);
    sw.Close();
    FileInfo fileinfo = new FileInfo(savefiledialog.FileName);
    fileName = fileinfo.Name;
    this.Text = fileName + " - " + programeName;
    textchanged = false;
    tSSL_change.Text = "";
}
else return;
}
private void Form1_Load(object sender, EventArgs e)
{
    textchanged = false;
    tSSL_change.Text = "";
    statusStrip.Visible = false;
    ToolStripMenuItem.Enabled = false;
    ToolStripMenuItem.Enabled = false;
    ToolStripMenuItem.Enabled = false;
    ToolStripMenuItem.Enabled = false;
    this.Text = fileName + " - " + programeName;
    timer.Start();
}
private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    if (textchanged == true)
    {
        DialogResult result = MessageBox.Show("? ", "notepad",
            MessageBoxButtons.YesNoCancel, MessageBoxIcon.Information);
        if (result == DialogResult.Yes)
        {
            saveFile();
            newFile();
        }
        else if (result == DialogResult.No)
        {
            newFile();
        }
    }
    else newFile();
}
private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    if (textchanged==true)
    {
        DialogResult result = MessageBox.Show("? ", "notepad",
            MessageBoxButtons.YesNoCancel, MessageBoxIcon.Information);
```

```

        if (result == DialogResult.Yes)
        {
            saveFile();
            openFile();
        }
        else if (result == DialogResult.No)
        {
            openFile();
        }
    }
    else openFile();
}

private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    if (!fileName.Equals(""))
    {
        saveFile();
    }
    else
    {
        saveAsFile();
    }
}

private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    saveAsFile();
}

private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    pageSetupDialog.Document = printDocument;
    this.pageSetupDialog.AllowMargins = true;
    this.pageSetupDialog.AllowOrientation = true;
    this.pageSetupDialog.AllowPaper = true;
    this.pageSetupDialog.AllowPrinter = true;
    this.pageSetupDialog.Document = this.printDocument;
    pageSetupDialog.ShowDialog();
}

private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    this.printDialog.Document = this.printDocument;
    this.printDialog.PrinterSettings
this.pageSetupDialog.PrinterSettings;
    if (this.printDialog.ShowDialog() == DialogResult.OK)
    {
        try
        {
            this.printDocument.Print();
        }
        catch (Exception ex)
        {

```

```
        MessageBox.Show(ex.Message, "notepad", MessageBoxButtons.OK,
        MessageBoxIcon.Error);
    }
}

private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    if (textchanged == true)
    {
        DialogResult result = MessageBox.Show("? ", "notepad",
        MessageBoxButtons.YesNoCancel, MessageBoxIcon.Information);
        if (result == DialogResult.Yes)
        {
            ToolStripMenuItem_Click(sender, e);
            Application.Exit();
        }
        else if (result == DialogResult.No)
        {
            Application.Exit();
        }
    }
    else
    {
        Application.Exit();
    }
}

private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    if (selectText.Equals(""))
    {
        ToolStripMenuItem.Enabled = false;
        ToolStripMenuItem.Enabled = false;
        ToolStripMenuItem.Enabled = false;
    }
}

private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    rTB_container.Undo();
}

private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    rTB_container.Copy();
}

private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    rTB_container.Paste();
}

private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    rTB_container.Cut();
}
```

```
}
private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    rTB_container.SelectedText = "";
}
private void ToolStripMenuItem_Click(object sender, EventArgs e)
{

}
private void ToolStripMenuItem_Click(object sender, EventArgs e)
{

}
private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    rTB_container.SelectAll();
}
private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    rTB_container.Text += DateTime.Now.ToString();
}
private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    if (ToolStripMenuItem.Checked == true)
    {
        ToolStripMenuItem.Checked = false;
        rTB_container.WordWrap = false;
    }
    else
    {
        ToolStripMenuItem.Checked = true;
        rTB_container.WordWrap = true;
    }
}
private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    FontDialog fontdialog = new FontDialog();
    if(fontdialog.ShowDialog() == DialogResult.OK)
    {
        rTB_container.Font = fontdialog.Font;
    }
}
private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    if (ToolStripMenuItem.Checked || rTB_container.Text.Equals(""))
    {
        ToolStripMenuItem.Enabled = false;
    }
    else
    {

```



```

        ToolStripMenuItem.Enabled = true;
    }
}
private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    if (ToolStripMenuItem.Checked == true)
    {
        ToolStripMenuItem.Checked = false;
        statusStrip.Visible = false;
    }
    else
    {
        ToolStripMenuItem.Checked = true;
        statusStrip.Visible = true;
    }
}
private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    if (ToolStripMenuItem.Checked == true)
    {
        ToolStripMenuItem.Checked = false;
        this.WindowState = FormWindowState.Normal;
        this.FormBorderStyle =
System.Windows.Forms.FormBorderStyle.Sizable;
    }
    else
    {
        ToolStripMenuItem.Checked = true;
        this.WindowState = FormWindowState.Maximized;
        this.FormBorderStyle =
System.Windows.Forms.FormBorderStyle.None;
    }
}
private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    System.Diagnostics.Process.Start(helpUrl);
}
private void ToolStripMenuItem_Click(object sender, EventArgs e)
{
    AboutBox1 about = new AboutBox1();
    about.Show();
    about.Owner = this;
}
private void rTB_container_TextChanged(object sender, EventArgs e)
{
    textchanged = true;
    tSSL_change.Text = "";
}
private void Form1_FormClosing(object sender, FormClosingEventArgs e)
{

```

```

        if (textchanged == true)
        {
            DialogResult result = MessageBox.Show("? ", "notepad",
                MessageBoxButtons.YesNoCancel, MessageBoxIcon.Information);
            if (result == DialogResult.Yes)
            {
                ToolStripMenuItem_Click(sender, e);
                e.Cancel = false;
            }
            else if (result == DialogResult.No)
            {
                e.Cancel = false;
            }
            else e.Cancel = true;
        }
        else Application.Exit();
    }

    private void timer_Tick(object sender, EventArgs e)
    {
        tSSL_time.Text = System.DateTime.Now.ToString();
        tSSL_charNum.Text = " "+rTB_container.Text.Length.ToString()+" ";
        int totalLine =
rTB_container.GetLineFromCharIndex(rTB_container.Text.Length) + 1
        int index = rTB_container.GetFirstCharIndexOfCurrentLine();
        int line = rTB_container.GetLineFromCharIndex(index)+1;
        int col = rTB_container.SelectionStart - index + 1; SelectionStart -
=
        tSSL_loc.Text = " "+line+" "+" "+col+" ";
    }
}

public string ClearSpe(string strSource)
{
    if (strSource.IndexOf(" ") > -1)
    {
        strSource = strSource.Replace(" ", "");
    }
    return strSource;
}

    FileStream F = new FileStream("test.dat", FileMode.OpenOrCreate,
FileAccess.ReadWrite);
    for (int i = 1; i <= 20; i++)
    {
        F.WriteByte((byte)i);
    }
    F.Position = 0;
    for (int i = 0; i <= 20; i++)
    {
        Console.Write(F.ReadByte() + " ");
    }
}

```

```
F.Close();
string[] names = new string[] { "Zara Ali", "Nuha Ali" };
using (StreamWriter sw = new StreamWriter("names.txt"))
{
    foreach (string s in names)
    {
        sw.WriteLine(s);
    }
}
string line = "";
using (StreamReader sr = new StreamReader("names.txt"))
{
    while ((line = sr.ReadLine()) != null)
    {
        Console.WriteLine(line);
    }
}
try
{
    BinaryWriter bw = new BinaryWriter(new FileStream("mydata",
    FileMode.Create));
    Int32 vInt = Convert.ToInt32(Console.ReadLine());
    Double vDoub = Convert.ToDouble(Console.ReadLine());
    String vStr = Console.ReadLine();
    bw.Write(vInt);
    bw.Write(vDoub);
    bw.Write(vStr);
    bw.Close();
}
catch (IOException e)
{
    Console.WriteLine(e.Message);
    return;
}
catch (Exception e)
{
    Console.WriteLine(e.Message);
    return;
}
try
{
    BinaryReader br = new BinaryReader(new FileStream("mydata",
    FileMode.Open));
    Console.WriteLine("Read: \n{0} \n{1} \n{2}", br.ReadInt32(),
    br.ReadDouble(), br.ReadString());
    br.Close();
}
catch (IOException e)
{
    Console.WriteLine(e.Message);
```

```

    }
    catch (Exception e)
    {
        Console.WriteLine(e.Message);
    }

    [DllImport("user32.dll")]
    public static extern UInt32 RegisterHotKey(IntPtr hWnd, UInt32 id, UInt32
fsModifiers, UInt32 vk);
    public Form1()
    {
        InitializeComponent();
        RegisterHotKey(this.Handle, 247696411, 0, (UInt32)Keys.F10);
    }
    protected override void WndProc(ref Message m)
    {
        const int WM_HOTKEY = 0x0312;
        if (m.Msg == WM_HOTKEY && m.WParam.ToInt32() == 247696411)
        {

            Start();
        }
        base.WndProc(ref m);
        public static System.Data.DataSet ExcelSqlConnection(string filepath,
string tableName)
        {
            string strCon = "Provider=Microsoft.Jet.OLEDB.4.0;Data Source=" +
filepath + ";Extended Properties='Excel 8.0;HDR=YES;IMEX=1'";
            OleDbConnection ExcelConn = new OleDbConnection(strCon);
            try
            {
                string strCom = string.Format("SELECT * FROM [Sheet1$]");
                ExcelConn.Open();
                OleDbDataAdapter myCommand = new OleDbDataAdapter(strCom,
ExcelConn);

                DataSet ds = new DataSet();
                myCommand.Fill(ds, "[" + tableName + "$]");
                ExcelConn.Close();
                return ds;
            }
            catch
            {
                ExcelConn.Close();
                return null;
            }
        }
    }
}

```